LAB 9
CSC 1310 FALL 2026
BINARY SEARCH TREES (BST)

Lab Assignment Policies

1. **PAY ATTENTION!!** You may not use generative AI **in any way** for Lab 9. Not for generating code, troubleshooting code, or providing example code.
2. **You must work alone on this assignment**. This is not a pair programming assignment.
3. You may not use the following in this assignment:
    a. while(true) or while(1)
    b. goto statements
    c. recursively calling main
    d. exit()
    e. ternary operators (any syntax requiring a '?')
    f. range-based for loops
    g. try-catch
    h. getline(cin>>ws, variable);
    i. lambda functions
    j. defining all programmer-defined functions above the main function
    k. code that has no comment block at the top indicating the filename, author, date, and purpose

    If you use any of the things listed above or use code that has not been taught in the class (without referencing your detailed source in a comment) then full points will be deducted for that criteria item in the rubric.

Description

Practice recursive search and tree height, then investigate how insertion order changes the structure and efficiency of a binary search tree.

Provided files

Download **lab9_given_code.zip**.

Open **Driver.cpp** and **BST.h** and look over the provided code.

The `Node` structure, `insert()`, `destroyTree()`, the main function in the Driver.cpp file, and output statements are provided.

Do not change these supplied functions unless instructed.

Your job in this assignment is to complete the `search()` and `height()` functions.

Part 1: Build and sketch a BST

The driver inserts these values in the order shown: 50, 30, 70, 20, 40, 60, 80, 10.

On a provided piece of paper, sketch the resulting tree before running your program.

Label each node with its value and show each left/right child.

Part 2: Recursive Search

Implement `bool search(Node* root, int target)` using recursion. You may use provided example programs (under the Module example programs) to help with implementation.

Return **`false`** for an empty subtree, **`true`** if the current node matches, and otherwise recurse into only the appropriate subtree based on the BST ordering rule.

Example Target (Key) for Testing	Expected Output
40	Found
10	Found
65	Not Found
90	Not Found

Part 3: Recursive Height

Implement `int height(Node* root)` recursively.

For this lab, height is the number of EDGES in the longest path from the root to a leaf where an empty tree is height of -1 and a single-node tree is height of 0. Compute the left and right subtree heights and return the larger plus one.

Expected height of the Part 1 tree is 3.

Part 4: Change Insertion Order

The provided driver deletes the first tree and builds a new tree by inserting: 10, 20, 30, 40, 50, 60, 70, 80.

Draw the resulting tree on the provided paper. Expected height: 7.

Answer the four questions on the provided paper.

Work on Program 3!

This lab assignment is shorter than most, so you should have about 20 to 30 minutes to work on your Program 3 assignment with your partner (or alone if you decided not to pair).

Lab Report

First, make sure to complete the lab report.
After you have completed this assignment, fill out the lab report and submit your lab report in your zip file along with your source files and images. Name your screenshot **lab9ReportProof**.
Lab Report Link: https://tntech.co1.qualtrics.com/jfe/form/SV_0iJcCTCBgguPDpQ

Submission

Submit the following to Lab 9 in Canvas:

1. Driver.cpp
2. BST.h (given file)
3. Lab9Worksheet_page1.png
4. Lab9Worksheet_page2.png
5. Lab9Report.png